

课程笔记

CS146S Week 7: Modern Software Support (Code Review)

基于 Mihail Eric + Tomas Reimers (Graphite CPO) 授课内容整理

2025 年 11 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 1 lecture (Mon)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 代码审查的价值与方法	2
1.1 为什么代码审查是最高杠杆的工程活动	2
1.2 代码审查应该关注什么	2
1.2.1 逻辑与正确性错误	2
1.2.2 可读性与可维护性	2
1.2.3 性能	2
1.2.4 安全	2
1.2.5 最佳实践	2
1.3 什么是好的代码审查	2
1.4 本章小结	3
2 AI 代码审查的新世界	3
2.1 主要工具	3
2.2 AI 代码审查带来的改变	3
2.3 当前的局限	3
2.4 本章小结	4
3 总结与延伸	4
3.1 关键点	4
3.2 拓展阅读	4

1 代码审查的价值与方法

1.1 为什么代码审查是最高杠杆的工程活动

代码审查 (Code Review) 是提升个人工程能力和团队代码质量最具杠杆效应的活动之一。数据佐证：

代码审查的量化价值

- 代码审查的错误检出率为 55–60%，而各种测试模式的检出率仅为 25–45%
- 无代码审查 vs 有代码审查的程序错误率：4.5 → 0.82 errors/100 lines
- AT&T 的研究显示：代码审查带来 14% 的生产力提升和 90% 的缺陷减少

来源：Coding Horror

1.2 代码审查应该关注什么

1.2.1 逻辑与正确性错误

代码是否按预期工作？是否有边界条件处理不当、off-by-one 错误等？

1.2.2 可读性与可维护性

代码是否易于理解和后续修改？命名是否清晰？函数职责是否单一？

1.2.3 性能

是否存在不必要的数据库查询、N+1 问题、不合理的时间/空间复杂度？

1.2.4 安全

是否存在 SQL injection、XSS、未验证的输入、敏感数据泄露等安全隐患？

1.2.5 最佳实践

是否遵循项目的编码规范、设计模式和架构约定？

1.3 什么是好的代码审查

“This won't work” 不是好的审查意见

简单的否定不提供任何教学价值。好的审查意见应该是建设性的、具体的，并邀请对话。例如：

“I see your new method matches the existing style in this file, taking [X] parameters. Having that many parameters hurts readability and implies the function is doing too much. What do you think about refactoring this method and the existing ones in a later pull request to reduce how many parameters they take?”

1.4 本章小结

代码审查是软件质量的核心保障，其错误检出率远超自动化测试。好的审查意见应该是具体的、建设性的，并为被审查者提供学习机会。

2 AI 代码审查的新世界

2.1 主要工具

- **Graphite**: 本周嘉宾 Tomas Reimers 是其 CPO
- **Greptile**: 基于代码库理解的 AI 审查
- **CodeRabbit**: 自动化 PR 审查
- **Claude Code / Codex**: 通用 AI 编码工具也可用于审查

2.2 AI 代码审查带来的改变

1. **效率提升**: 自动化处理常规检查，缩短审查周期
2. **一致性**: AI 对所有代码应用相同的标准
3. **知识共享**: AI 可以携带整个代码库的上下文
4. **减轻认知负担**: 让人类专注于复杂逻辑和架构决策
5. **持续改进**: AI 系统随时间积累更多数据而改善
6. **整体理解**: AI 可以从全局视角审视变更的影响

AI 代码审查的分工模式

最佳实践不是用 AI 完全替代人类审查，而是让 AI 处理“表层”问题（格式、命名、常见 bug 模式、安全检查），让人类专注于“深层”问题（架构合理性、业务逻辑正确性、长期维护性）。

2.3 当前的局限

- **配置和设置成本**: 需要初始投入来训练系统
- **假阳性**: 需要持续训练系统以减少误报
- **无法捕获代码库的惯用法和最佳实践**: 仓库特有的 idiom 仍然超出 AI 能力
- **无法处理复杂的业务逻辑和架构决策**: 这正是人类仍然不可或缺的地方
- **安全变更需格外谨慎**: AI 可能遗漏微妙的安全影响
- **经常遗漏边界情况**

AI 时代代码审查更重要了，而不是更不重要

当 AI 编写了你的大部分代码时，代码审查变得**更加**重要。你拥有被合并和部署的代码——不能把锅甩给 AI。“The AI wrote it” 不是一个可接受的借口。

2.4 本章小结

AI 代码审查工具通过自动化常规检查显著提升了效率和一致性，但复杂的业务逻辑、架构决策和仓库特有的最佳实践仍然需要人类审查者。在 AI 编码时代，代码审查的重要性不降反升。

3 总结与延伸

Week 7 探讨了代码审查在 AI 编码时代的角色演变。核心信息是：AI 让代码审查更高效，但同时也让代码审查更加不可或缺——因为你需要审查的不仅是人写的代码，还有 AI 写的代码。

3.1 关键点

- 代码审查的错误检出率（55–60%）远超测试（25–45%）
- 好的审查意见是建设性的、具体的，并邀请对话
- 审查应覆盖五个维度：正确性、可读性、性能、安全、最佳实践
- AI 代码审查适合处理“表层”问题，人类专注“深层”问题
- 当前局限：假阳性、无法捕获 repo-specific idioms、无法处理复杂业务逻辑
- AI 时代代码审查更重要，“AI wrote it”不是免责借口

3.2 拓展阅读

- Graphite: <https://graphite.dev>
- Greptile: <https://greptile.com>
- CodeRabbit: <https://coderabbit.ai>
- Google 的代码审查最佳实践: <https://google.github.io/eng-practices/review/>
- “How to do a code review” by Blake Smith